

Day 14 — Introduction to PyTorch

Tensors · Autograd · nn.Module · Training Loop

Target: Google · Meta · OpenAI · Anthropic

The Big Picture

Yesterday you built a neural network entirely from scratch in NumPy. You wrote every forward pass, every gradient, every weight update by hand. Today you rebuild the same network in PyTorch and see exactly what PyTorch automates — and more importantly, why it exists.

PyTorch does not replace your understanding of backprop. It automates the tedious parts so you can focus on architecture and research. Engineers who understand Day 13 will use PyTorch far more effectively than those who skipped it.

File	What it does
day14_tensors.py	Warm-up — PyTorch tensors, operations, GPU movement
day14_autograd.py	Warm-up — automatic differentiation, gradients
day14_neural_network.py	Rebuild Day 13 network in PyTorch — compare results

What Is PyTorch?

PyTorch is a deep learning framework built by Meta (Facebook AI Research). It is the dominant framework at every top AI lab — Google DeepMind, Anthropic, OpenAI, and Meta all use it. Understanding PyTorch deeply is non-negotiable for ML/AI engineering roles.

PyTorch component	What it replaces from Day 13	Why it is better
torch.Tensor	np.ndarray	Runs on GPU, tracks gradients automatically
autograd	Your manual backprop code	Computes any gradient automatically via chain rule
nn.Module	Your manual weight matrices	Organises layers, parameters, forward pass cleanly
nn.Linear	$X @ W + b$ by hand	Handles weight init, matrix multiply, bias automatically
optim.SGD / Adam	Your manual $W -= lr * dW$	Optimiser handles all weight updates in one line
nn.BCELoss	Your manual <code>bce()</code> function	Numerically stable, handles edge cases automatically

Concept: Tensors — PyTorch's Version of NumPy Arrays

A tensor is like a NumPy array but with two superpowers: it can run on a GPU (massively faster for matrix operations), and it can track every operation performed on it so PyTorch can automatically compute gradients.

Creating tensors:

```
import torch
import numpy as np

# From a Python list
t1 = torch.tensor([1.0, 2.0, 3.0])
print(t1)          # tensor([1., 2., 3.])
print(t1.dtype)    # torch.float32

# From NumPy array
arr = np.array([1.0, 2.0, 3.0])
t2 = torch.from_numpy(arr)  # shares memory with arr
t3 = torch.tensor(arr)      # makes a copy

# Common creation functions – same as NumPy
zeros = torch.zeros(3, 4)    # shape (3, 4) of zeros
ones  = torch.ones(3, 4)     # shape (3, 4) of ones
randn = torch.randn(3, 4)    # random normal values
arange = torch.arange(0, 10, 2) # [0, 2, 4, 6, 8]
```

Tensor shapes and operations:

```
x = torch.randn(3, 4)  # shape (3, 4)

# Shape inspection
print(x.shape)          # torch.Size([3, 4])
print(x.shape[0])       # 3
print(x.numel())        # 12 – total number of elements

# Operations – same as NumPy
y = torch.randn(4, 5)
z = x @ y               # matrix multiply – shape (3, 5)
z = x + 1               # broadcast add
z = x * 2               # element-wise multiply
z = x.T                # transpose – shape (4, 3)

# Reshape
flat = x.reshape(-1)    # flatten to 1D – shape (12,)
back = flat.reshape(3, 4) # back to (3, 4)

# Converting back to NumPy
arr = x.detach().numpy() # .detach() removes gradient tracking first
```

Moving to GPU:

One of the main reasons PyTorch exists — one line moves all computation to GPU.

```
# Check if GPU is available
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {device}')

# Move tensor to GPU
x = torch.randn(3, 4)
x = x.to(device)      # now runs on GPU if available
```

```
# Move model to GPU (Day 15 onwards)
model = model.to(device)
inputs = inputs.to(device)
```

Concept: Autograd — Automatic Differentiation

Autograd is PyTorch's automatic differentiation engine. It tracks every operation you perform on tensors and can automatically compute the gradient of any value with respect to any other value. This replaces all your manual backprop code from Day 13.

How autograd works:

When you create a tensor with `requires_grad=True`, PyTorch builds a computational graph as you perform operations. When you call `.backward()`, it traverses this graph backwards and computes all gradients automatically using the chain rule.

```
import torch

# requires_grad=True tells PyTorch to track this tensor
x = torch.tensor(3.0, requires_grad=True)

# Build a computation
y = x ** 2 + 2 * x + 1  # y = x^2 + 2x + 1

# Compute gradient dy/dx
y.backward()

# dy/dx = 2x + 2 = 2(3) + 2 = 8
print(x.grad)  # tensor(8.) - PyTorch computed this automatically
```

Gradient accumulation — important:

PyTorch accumulates (adds) gradients by default. You must zero them out before each backward pass or they stack up incorrectly.

```
# WRONG - gradients accumulate across iterations
for epoch in range(100):
    loss = compute_loss()
    loss.backward()  # gradients keep adding up - wrong

# CORRECT - zero gradients before each backward pass
for epoch in range(100):
    optimizer.zero_grad()  # clear gradients from last iteration
    loss = compute_loss()
    loss.backward()  # compute fresh gradients
    optimizer.step()  # update weights
```

`torch.no_grad()` — turning off gradient tracking:

During evaluation you do not need gradients. Wrapping evaluation in `torch.no_grad()` saves memory and speeds up inference.

```
# During training – needs gradients
output = model(X_train)

# During evaluation – no gradients needed
with torch.no_grad():
    output = model(X_test)  # faster, uses less memory
```

Concept: nn.Module — Building Neural Networks

`nn.Module` is the base class for all neural networks in PyTorch. You define your network by subclassing it, specifying the layers in `__init__`, and defining the forward pass in `forward()`. PyTorch handles `backward()` automatically via `autograd`.

The structure of every PyTorch model:

```
import torch
import torch.nn as nn

class MyNetwork(nn.Module):
    def __init__(self) -> None:
        super().__init__()  # always call this first
        # Define layers here – PyTorch tracks their parameters automatically
        self.layer1 = nn.Linear(4, 8)  # input_size=4, output_size=8
        self.layer2 = nn.Linear(8, 4)
        self.layer3 = nn.Linear(4, 1)
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # Define the forward pass – autograd builds the graph here
        x = self.relu(self.layer1(x))  # layer1 + relu
        x = self.relu(self.layer2(x))  # layer2 + relu
        x = self.sigmoid(self.layer3(x)) # layer3 + sigmoid
        return x

model = MyNetwork()
print(model)  # shows architecture
```

nn.Linear — what it does:

`nn.Linear(in, out)` creates a linear layer that computes $X @ W.T + b$. It automatically initialises `W` and `b`, tracks them as parameters, and includes them in gradient updates.

```
layer = nn.Linear(4, 8)
# layer.weight shape: (8, 4) – PyTorch stores it transposed
# layer.bias shape: (8,)

# Counting parameters
total = sum(p.numel() for p in model.parameters())
```

```
print(f'Total parameters: {total}')
```

Concept: The PyTorch Training Loop

Every PyTorch training loop follows the same 5-step pattern. Memorise this — you will write it hundreds of times.

Step	Code	What it does
1. Zero gradients	<code>optimizer.zero_grad()</code>	Clear gradients from last iteration
2. Forward pass	<code>output = model(X)</code>	Run input through network, build graph
3. Compute loss	<code>loss = criterion(output, y)</code>	Measure how wrong predictions are
4. Backward pass	<code>loss.backward()</code>	Compute all gradients via autograd
5. Update weights	<code>optimizer.step()</code>	Apply gradient descent to all parameters

```
# The complete training loop pattern
for epoch in range(epochs):
    # 1. Zero gradients – ALWAYS first
    optimizer.zero_grad()

    # 2. Forward pass
    output = model(X_train)

    # 3. Compute loss
    loss = criterion(output, y_train)

    # 4. Backward pass – autograd computes all gradients
    loss.backward()

    # 5. Update weights – optimiser applies gradient descent
    optimizer.step()

    if (epoch + 1) % 100 == 0:
        print(f'Epoch {epoch+1} | Loss: {loss.item():.4f}')
        # .item() converts single-value tensor to Python float
```

Day 13 vs Day 14 — Side by Side

Same network, same data, same task. See exactly what PyTorch replaces.

Task	Day 13 NumPy	Day 14 PyTorch
Define layers	6 weight matrices by hand	<code>nn.Linear</code> layers in <code>__init__</code>
Forward pass	<code>Z1 = X @ W1 + b1, A1 = relu(Z1)...</code>	<code>x = self.relu(self.layer1(x))</code>
Compute loss	Manual <code>bce()</code> function	<code>nn.BCELoss()</code>
Backprop	20+ lines of gradient math	<code>loss.backward()</code>

Weight update	6 lines of $W := lr * dW$	<code>optimizer.step()</code>
Lines of code	~80 lines	~40 lines
What you understand	Everything	Everything — because of Day 13

PyTorch does not make you smarter — it makes you faster. The engineer who built it from scratch in Day 13 will debug PyTorch problems that others cannot, because they know what is happening under the hood.

Warm-Up 1 — day14_tensors.py

```
import torch
import numpy as np

print("=== Tensor Creation ===")
t1 = torch.tensor([1.0, 2.0, 3.0, 4.0])
print(f"From list:    {t1}")
print(f"dtype:       {t1.dtype}")
print(f"shape:        {t1.shape}")
print()

print("=== Common Operations ===")
a = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
b = torch.tensor([[5.0, 6.0], [7.0, 8.0]])
print(f"a + b =\n{a + b}")
print(f"a @ b =\n{a @ b}")
print(f"a.T =\n{a.T}")
print()

print("=== Shapes ===")
x = torch.randn(3, 4)
print(f"shape:    {x.shape}")
print(f"numel:    {x.numel()}")
print(f"reshape: {x.reshape(2, 6).shape}")
print(f"flatten: {x.reshape(-1).shape}")
print()

print("=== NumPy Bridge ===")
arr = np.array([1.0, 2.0, 3.0])
t = torch.from_numpy(arr)
print(f"numpy -> tensor: {t}")
print(f"tensor -> numpy: {t.detach().numpy()}")
print()

print("=== Device ===")
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using: {device}")
x = torch.randn(3, 4).to(device)
print(f"Tensor on {device}: {x.shape}")
```

Warm-Up 2 — day14_autograd.py

```
import torch

print("=== Basic Autograd ===")
x = torch.tensor(3.0, requires_grad=True)
y = x ** 2 + 2 * x + 1
y.backward()
print(f"x = {x.item()}")
print(f"y = x^2 + 2x + 1 = {y.item()}")
print(f"dy/dx = 2x + 2 = {x.grad.item()} (expected: 8.0)")
print()
```

```

print("=== Vector Autograd ===")
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
y = (x ** 2).sum() # sum so we can call backward()
y.backward()
print(f"x      = {x.data}")
print(f"dy/dx = 2x = {x.grad} (expected: [2, 4, 6])")
print()

print("=== Gradient Accumulation Warning ===")
x = torch.tensor(2.0, requires_grad=True)
for i in range(3):
    y = x * 2
    y.backward()
    print(f" Iteration {i+1}: x.grad = {x.grad.item()} (accumulates!)")

# Reset and do it correctly
x = torch.tensor(2.0, requires_grad=True)
print("Correct way:")
for i in range(3):
    if x.grad is not None:
        x.grad.zero_() # zero gradient before each backward
    y = x * 2
    y.backward()
    print(f" Iteration {i+1}: x.grad = {x.grad.item()} (always 2.0)")
print()

print("=== no_grad context ===")
x = torch.randn(3, 3, requires_grad=True)
with torch.no_grad():
    y = x * 2
print(f"requires_grad inside no_grad: {y.requires_grad}") # False

```


Full Network — day14_neural_network.py

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
from sklearn.model_selection import train_test_split

# CREATE DATA – same as Day 13
np.random.seed(42)
n = 500
age = np.random.randint(22, 60, n).astype(float)
income = np.random.randint(20000, 100000, n).astype(float)
education = np.random.randint(8, 22, n).astype(float)
experience = np.random.randint(0, 20, n).astype(float)
hired = (0.3 * education + 0.5 * experience
         + np.random.normal(0, 2, n) > 10).astype(float)

X_np = np.column_stack([age, income, education, experience])
y_np = hired.reshape(-1, 1)

# Normalise
X_np = (X_np - X_np.mean(axis=0)) / X_np.std(axis=0)

# Split
X_tr, X_te, y_tr, y_te = train_test_split(X_np, y_np, test_size=0.2, random_state=42)

# Convert to PyTorch tensors – neural networks need float32
X_train = torch.tensor(X_tr, dtype=torch.float32)
X_test = torch.tensor(X_te, dtype=torch.float32)
y_train = torch.tensor(y_tr, dtype=torch.float32)
y_test = torch.tensor(y_te, dtype=torch.float32)

# DEFINE NETWORK
class HiringNet(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        # Same architecture as Day 13: 4 -> 8 -> 4 -> 1
        self.layer1 = nn.Linear(4, 8)
        self.layer2 = nn.Linear(8, 4)
        self.layer3 = nn.Linear(4, 1)
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.relu(self.layer1(x)) # 4 -> 8 -> relu
        x = self.relu(self.layer2(x)) # 8 -> 4 -> relu
        x = self.sigmoid(self.layer3(x)) # 4 -> 1 -> sigmoid
        return x

# SETUP
torch.manual_seed(42)
model = HiringNet()
criterion = nn.BCELoss() # binary cross-entropy
optimizer = optim.SGD(model.parameters(), lr=0.01) # gradient descent
```

```

# Count parameters
total_params = sum(p.numel() for p in model.parameters())
print(f"Total parameters: {total_params}")
print(f"Architecture: {model}")
print()

# TRAINING LOOP
print("Training...")
for epoch in range(1000):
    # 1. Zero gradients – always first
    optimizer.zero_grad()

    # 2. Forward pass
    output = model(X_train)

    # 3. Compute loss
    loss = criterion(output, y_train)

    # 4. Backward pass – autograd computes all gradients
    loss.backward()

    # 5. Update weights
    optimizer.step()

    if (epoch + 1) % 100 == 0:
        print(f" Epoch {epoch+1:4d} | Loss: {loss.item():.4f}")

# EVALUATE
model.eval() # set model to evaluation mode
with torch.no_grad():
    predictions = model(X_test)
    predicted    = (predictions > 0.5).float()
    accuracy     = (predicted == y_test).float().mean().item()

print(f"\nTest accuracy: {accuracy:.3f}")
print(f"Correct: {int(accuracy * len(y_test))} / {len(y_test)}")

```

Expected Results

Metric	Expected	Notes
Total parameters	81	Same as Day 13 — same architecture
Loss at epoch 100	~0.45-0.55	PyTorch uses He init by default in nn.Linear
Loss at epoch 1000	~0.32-0.40	Similar to Day 13
Test accuracy	~80-85%	Similar to Day 13 result

nn.Linear uses Kaiming (He) initialisation by default — that is why it trains well without any special setup. This is one of the many things PyTorch handles for you.

New Code Concepts — Summary

Concept	What it does
<code>torch.tensor(data, dtype=torch.float32)</code>	Creates a tensor from Python list or NumPy array
<code>tensor.to(device)</code>	Moves tensor to CPU or GPU
<code>requires_grad=True</code>	Tells PyTorch to track this tensor for gradient computation
<code>loss.backward()</code>	Computes all gradients automatically via chain rule
<code>optimizer.zero_grad()</code>	Clears accumulated gradients — call before every backward()
<code>optimizer.step()</code>	Updates all weights using computed gradients
<code>model.eval()</code>	Sets model to evaluation mode — disables dropout, batchnorm updates
<code>torch.no_grad()</code>	Context manager — disables gradient tracking for speed
<code>loss.item()</code>	Converts single-value tensor to Python float for printing
<code>p.numel()</code>	Returns total number of elements in a tensor

Anki Cards — Day 14

Q: What are the 5 steps of every PyTorch training loop?

A: `zero_grad`, forward pass, compute loss, backward, `optimizer.step`

Q: Why must you call `optimizer.zero_grad()` first?

A: PyTorch accumulates gradients — not zeroing causes incorrect gradient buildup

Q: What does `loss.backward()` do?

A: Computes all gradients automatically via autograd and the chain rule

Q: What does `optimizer.step()` do?

A: Updates all model weights using the computed gradients

Q: What does `model.eval()` do?

A: Sets model to evaluation mode — disables dropout and batchnorm training behaviour

Q: What does `torch.no_grad()` do?

A: Disables gradient tracking — faster inference, less memory

Q: What does `nn.Linear(4, 8)` create?

A: A linear layer: input size 4, output size 8 — computes $X @ W.T + b$

Q: What is `requires_grad=True`?

A: Tells PyTorch to track this tensor in the computational graph for autograd

Q: What does `.item()` do on a tensor?

A: Converts a single-value tensor to a Python float

Q: What initialisation does `nn.Linear` use by default?

A: Kaiming (He) initialisation — same as what we used manually in Day 13

Next: Day 15 — Deeper PyTorch. DataLoader, training on real datasets, saving and loading models.